Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

3. Thiết kế dịch vụ & API 2
3.1. REST API Design — nguyên tắc cho Microservices 3
3.1.1. Tại sao API design quan trọng hơn trong microservices? 3
3.1.2. Nguyên tắc thiết kế REST API 4
3.2. API Versioning & Schema Evolution 7
3.2.1. Các chiến lược versioning 7
3.3. OpenAPI & API Documentation 9
3.4. DTO Pattern — tách Internal Model và External Contract 11
3.4.1. DTO (Data Transfer Object) 11
3.4.2. Lợi ích thực tế 13
3.5. Hexagonal Architecture — tách biệt lõi nghiệp vụ khỏi hạ tầng 13
3.6. Contract Testing — kiểm thử hợp đồng API 14
3.6.1. Vấn đề: API thay đổi làm lỗi máy khách (crash consumer) 14
3.6.2. Consumer-Driven Contract Testing 15
3.6.3. Áp dụng cho KBM 16
3.7. GraphQL — khi REST không đủ linh hoạt 16
3.7.1. REST vs GraphQL — so sánh 17
3.7.2. N+1 Problem — thách thức chính của GraphQL 17
3.7.3. Khi nào chọn REST, khi nào chọn GraphQL? 18
3.8. Ranh giới của chương này 19
3.9. Case study: KBM 19
3.9.1. Hiện trạng 20
3.9.2. Bài học rút ra 20
3.9.3. Phục vụ nhiều loại client và tích hợp judge ngoài — bài học mở rộng 21
3.10. Tổng kết 22
3.11. Đọc thêm 22
Tài liệu tham khảo 23

CHƯƠNG 3

Thiết kế dịch vụ & API

> "Public APIs, like diamonds, are forever. You have one chance to get it right so give it your best."
> — Joshua Bloch, "How to Design a Good API and Why it Matters" (2006)

MỤC TIÊU HỌC TẬP

- Nắm vững nguyên tắc thiết kế REST API cho microservices
- Hiểu các chiến lược API versioning và schema evolution
- Sử dụng OpenAPI/Swagger để tạo tài liệu API tự động
- Áp dụng DTO pattern và Hexagonal Architecture (Ports & Adapters) để tách biệt lõi nghiệp vụ khỏi hợp đồng và hạ tầng bên ngoài
- Hiểu Contract Testing (Pact, Spring Cloud Contract) để ngăn breaking change
- So sánh REST và GraphQL, nhận diện bài toán phù hợp cho từng công nghệ
- Phân tích bài học thực tế từ API design trong KBM

3.1. REST API Design — nguyên tắc cho Microservices

Hãy tưởng tượng REST API như những mẫu đơn chuẩn hóa của phòng đào tạo. Nếu mỗi khoa yêu cầu một biểu mẫu khác nhau, sinh viên sẽ gặp khó khăn. Trong hệ thống phân tán, một thiết kế API chuẩn mực là yêu cầu tiên quyết để các dịch vụ giao tiếp.

3.1.1. Tại sao API design quan trọng hơn trong microservices?

Trong monolith, giao tiếp giữa các module là gọi hàm nội bộ (in-process call) — nếu interface thay đổi, trình biên dịch (compiler) sẽ báo lỗi ngay. Trong microservices, giao tiếp diễn ra qua mạng lưới (network) (HTTP, gRPC, messaging) — và **không có trình biên dịch nào phát hiện lỗi khi API thay đổi**. Một thay đổi phá vỡ cấu trúc (breaking change) trong API có thể gây gián đoạn cho các dịch vụ khác mà không ai hay biết, cho đến khi hệ thống trên môi trường thực tế (production) gặp sự cố.

Do đó, API trong microservices cần được thiết kế **cẩn thận hơn**, **ổn định hơn**, và **có chiến lược evolution rõ ràng** — đây không phải tùy chọn, mà là điều kiện tiên quyết.

Richardson Maturity Model. Leonard Richardson[1] đề xuất mô hình 4 cấp độ trưởng thành cho REST API [2a, Ch.3]:



Hình 3.1: Richardson Maturity Model — bốn cấp độ trưởng thành của REST API

Trong bốn cấp độ này, hầu hết microservices thực tế (bao gồm KBM LMS) dừng lại ở **Level 2** — sử dụng đúng resource và HTTP verb, chưa cần đến hypermedia.

Level 3 (HATEOAS) — Khi nào cần? HATEOAS (*Hypermedia as the Engine of Application State*) yêu cầu phản hồi (response) chứa các liên kết (link) điều hướng — máy khách không cần biết trước cấu trúc URL, chỉ cần đi theo các liên kết từ phản hồi. Ví dụ: `GET /questions/123` trả về `{..., "_links": {"submissions": "/questions/123/submissions", "edit": "/questions/123"}}`. Ưu điểm: API có khả năng tự mô tả (self-documenting), máy khách (client) không phụ thuộc vào cấu trúc URL. Nhược điểm: làm tăng payload size, làm phức tạp response format, hầu hết frontend framework (React, Angular) không sử dụng tối đa HATEOAS. Trong thực tế, HATEOAS hiếm khi được áp dụng cho các API nội bộ (internal API) [4a, Ch.4] — phù hợp hơn cho các API công khai

[1] Cần lưu ý phân biệt Leonard Richardson (người đề xuất mô hình này) với Chris Richardson (tác giả của cuốn sách "Microservices Patterns" được trích dẫn nhiều trong tài liệu này).

(public API) khi provider không kiểm soát consumer — nhưng ngay cả GitHub API v3 cũng sử dụng cơ chế đánh phiên bản tường minh (explicit versioning) thay vì HATEOAS thuần túy. HATEOAS vẫn là ngoại lệ trong thực tế, không phải chuẩn phổ biến.

3.1.2. Nguyên tắc thiết kế REST API

1. Resource-oriented URLs — URL đại diện cho resource (danh từ), không phải hành động (động từ):

```
ĐÚNG: GET  /questions/{id}          → Lấy câu hỏi
ĐÚNG: POST /questions               → Tạo câu hỏi mới
ĐÚNG: GET  /contests/{id}/questions  → Câu hỏi trong cuộc thi

SAI:  GET  /getQuestion?id=123      → Dùng verb, query string
SAI:  POST /createQuestion           → URL chứa action
```

Tên phòng ban và danh từ

Hãy tưởng tượng hệ thống biển báo trong trường đại học. Chúng ta sẽ thấy biển báo chỉ đường đến "Phòng Đào tạo", "Thư viện" (danh từ — resource) chứ không phải biển báo "Đăng ký môn học", "Mượn sách" (động từ — action). REST API cũng vậy: URL nên định danh "nơi chứa tài nguyên" thay vì "hành động sẽ thực hiện".

2. HTTP methods đúng ngữ nghĩa:

Các phương thức HTTP cốt lõi và ý nghĩa tương ứng bao gồm:

- `GET`: Được sử dụng để đọc hoặc lấy thông tin tài nguyên (ví dụ: `GET /questions/123`). Theo RFC 9110, `GET` vừa mang tính **safe** (chỉ đọc, không làm thay đổi trạng thái tài nguyên trên server) vừa mang tính **idempotent** (gọi lặp lại $N \geq 1$ lần có tác động lên trạng thái hệ thống giống hệt gọi một lần — mọi phương thức safe đều tự động idempotent, trong khi `PUT` và `DELETE` ở dưới là idempotent nhưng không safe).
- `POST`: Dùng để tạo ra một tài nguyên hoàn toàn mới trên server (ví dụ: `POST /submissions`). Đặc điểm của `POST` là mặc định không mang tính idempotent.
- `PUT`: Có nhiệm vụ cập nhật hoặc ghi đè toàn bộ thông tin của một tài nguyên (ví dụ: `PUT /questions/123`). Quá trình này được thiết kế để luôn có tính idempotent.
- `PATCH`: Cho phép cập nhật một phần nhỏ dữ liệu của tài nguyên thay vì toàn bộ (ví dụ: `PATCH /users/123`). Theo tiêu chuẩn RFC 5789, `PATCH` mặc định là *non-idempotent*, mặc dù trong thực tế các lập trình viên thường chủ động thiết kế và triển khai nó một cách idempotent.
- `DELETE`: Có chức năng loại bỏ hoặc xóa một tài nguyên cụ thể (ví dụ: `DELETE /questions/123`). Dù thao tác xóa diễn ra thành công ở lần đầu hay báo lỗi không tìm thấy ở các lần sau, trạng thái dữ liệu vẫn nhất quán nên nó mang tính idempotent.

Bên cạnh việc sử dụng đúng phương thức HTTP để biểu diễn thao tác, máy chủ còn cần phản hồi kết quả bằng mã trạng thái chuẩn xác. Các mã trạng thái (status code) giúp client nhanh chóng hiểu được kết quả của một yêu cầu mà không cần phải phân tích toàn bộ nội dung phản hồi.

3. Response codes có ý nghĩa:

Hệ thống nên trả về các mã trạng thái tiêu chuẩn để biểu thị chính xác kết quả xử lý của máy chủ.

Bảng 3.1: Các mã trạng thái HTTP thường dùng trong microservices

Code	Khi nào dùng	Ví dụ
`200 OK`	Request thành công	`GET /questions/123` trả về question
`201 Created`	Tạo resource mới thành công	`POST /questions` trả về question vừa tạo
`204 No Content`	Thành công nhưng không có body	`DELETE /questions/123`
`400 Bad Request`	Input không hợp lệ	Thiếu field bắt buộc
`401 Unauthorized`	Chưa xác thực	Token hết hạn
`403 Forbidden`	Không có quyền	Student truy cập admin API
`404 Not Found`	Resource không tồn tại	`GET /questions/99999`
`409 Conflict`	Conflict trạng thái	Submission đã được chấm
`500 Internal Server Error`	Lỗi server	Database connection failed

4. Plural nouns, consistent naming — Luôn dùng số nhiều cho collection: `/questions`, không phải `/question`. Chọn một quy ước (kebab-case, camelCase, snake_case) và giữ nhất quán xuyên suốt.

5. Pagination, filtering, sorting — Mọi endpoint trả về collection phải hỗ trợ phân trang. Có hai chiến lược chính:

Offset-based (ví dụ: `?page=2&size=20`). Chiến lược này rất đơn giản và cho phép người dùng nhảy trực tiếp đến một trang cụ thể (jump to page). Tuy nhiên, hiệu năng sẽ giảm đáng kể ở các trang có số thứ tự lớn do cơ sở dữ liệu phải bỏ qua một lượng lớn bản ghi (skip N rows). Hơn nữa, kết quả có thể không ổn định nếu dữ liệu bị thêm hoặc xóa trong quá trình người dùng đang duyệt qua các trang.

Cursor-based (ví dụ: `?after=abc123&limit=20`). Cung cấp hiệu năng ổn định (độ phức tạp `O(1)` nếu được đánh index đúng cách) và đảm bảo kết quả nhất quán ngay cả khi dữ liệu thay đổi. Đổi lại, chiến lược này không hỗ trợ việc nhảy trang tự do và việc triển khai cũng phức tạp hơn nhiều so với offset-based.

KBM sử dụng **offset-based** (Spring Data `Pageable`) — hợp lý cho danh sách câu hỏi quy mô vừa. Với các tập dữ liệu (dataset) lớn (lịch sử nộp bài, nhật ký hệ thống), cursor-based pagination hiệu quả hơn khi `page > 100`.

Phản hồi phân trang (response pagination) nên bao gồm siêu dữ liệu (metadata) để máy khách nhận biết còn dữ liệu tiếp theo hay không:

Listing 3.1: Phân trang response kèm metadata

```
GET /questions?page=0&size=20&sort=createdAt,desc&difficulty=HARD

// Response envelope
{
  "data": [...],
  "pagination": {
    "page": 0,
    "size": 20,
    "totalElements": 157,
    "totalPages": 8,
```

```
    "hasNext": true
  }
}
```

6. Standardized Error Response — Các phản hồi lỗi (error response) cần có định dạng nhất quán để máy khách có thể xử lý tự động qua mã nguồn (programmatically). Lấy cảm hứng từ chuẩn Problem Details for HTTP APIs (RFC 9457 — bản thay thế RFC 7807), một error response nên bao gồm các trường sau. Lưu ý theo chuẩn gốc, các trường này đều tùy chọn ở mức từng response (`type` mặc định là `about:blank`); mức "Bắt buộc" dưới đây là *quy ước nội bộ KBM* áp đặt để client xử lý thống nhất:

- `type` **(Bắt buộc)**: URI định danh loại lỗi.
- `title` **(Bắt buộc)**: Mô tả ngắn gọn về loại lỗi.
- `status` **(Bắt buộc)**: Mã trạng thái HTTP chuẩn (HTTP status code).
- `detail` **(Khuyến nghị)**: Mô tả chi tiết nguyên nhân gây ra lỗi cụ thể này.
- `instance` **(Khuyến nghị)**: URI của request cụ thể đã gây ra lỗi.
- `errorCode` **(Bắt buộc — extension KBM)**: mã máy đọc được, ổn định, dạng chuỗi có namespace, ví dụ `VALIDATION_FAILED`, `JUDGE_UNAVAILABLE`. Không dùng số trần thiếu ngữ nghĩa.
- `traceId` **(Bắt buộc khi tracing hoạt động — extension KBM)**: W3C trace ID của occurrence; giúp client báo cáo cho đội ngũ hỗ trợ.
- `errors` **(Validation-only extension KBM — tùy chọn)**: member mở rộng do KBM tự định nghĩa theo cơ chế extension của RFC 9457; chứa danh sách các trường bị lỗi trong quá trình kiểm tra tính hợp lệ của dữ liệu (validation error), mỗi item dùng `pointer` (JSON Pointer), `detail`, và `code` tùy chọn.

Dưới đây là một ví dụ minh họa cho cấu trúc phản hồi lỗi theo chuẩn RFC 9457, giúp việc xử lý lỗi phía client trở nên đồng nhất hơn:

Listing 3.2: *Response lỗi chuẩn hóa (RFC 9457 Problem Details)*

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "urn:kbm:problem:validation-failed",
  "title": "Input validation failed",
  "status": 422,
  "detail": "The request contains invalid fields.",
  "instance": "urn:kbm:problem-instance:req-7f7bb1",
  "errorCode": "VALIDATION_FAILED",
  "traceId": "0af7651916cd43dd8448eb211c80319c",
  "errors": [
    {"pointer": "#/title", "detail": "must not be blank", "code": "NOT_BLANK"},
    {"pointer": "#/difficulty", "detail": "must be EASY, MEDIUM, or HARD", "code":
"INVALID_ENUM"}
  ]
}
```

Mọi dịch vụ trong hệ thống nên trả về lỗi theo cùng một định dạng — máy khách chỉ cần xây dựng **một** bộ xử lý lỗi (error handler) duy nhất. Trong Spring Boot, `@ControllerAdvice` + `GlobalExceptionHandler` giải quyết: mọi exception được bắt tại một điểm duy nhất và chuyển đổi thành định dạng phản hồi (response format) chuẩn (xem Bài học rút ra, §3.9.2).

7. Idempotency — Một số thao tác (operation) cần đảm bảo tính **idempotent**: thực hiện nhiều lần có cùng tác động lên trạng thái hệ thống như thực hiện một lần. GET, PUT, DELETE vốn dĩ đã mang tính idempotent. POST thì không — nếu máy khách gọi `POST /submissions` hai lần (do request bị gửi lại khi gặp lỗi mạng — network retry), hệ thống có thể tạo ra hai bản ghi submission.

Giải pháp: **Idempotency Key** — máy khách gửi kèm header `Idempotency-Key: <UUID>`. Máy chủ kiểm tra khóa (key) đã được xử lý chưa: nếu rồi, trả kết quả cũ; nếu chưa, xử lý và lưu khóa. Stripe, PayPal đều dùng mẫu thiết kế (pattern) này cho các API thanh toán — điều này đặc biệt quan trọng (critical) khi liên quan đến tài chính hoặc các thao tác nghiệp vụ cốt lõi (business operation).

3.2. API Versioning & Schema Evolution

Tại sao cần versioning? Trong monolith, thay đổi internal API chỉ yêu cầu tái cấu trúc mã nguồn (refactor code) và chạy lại các bài kiểm thử (test). Trong microservices, service A thay đổi định dạng phản hồi (response format) → service B (consumer) *có thể* gặp lỗi nghiêm trọng (crash) nếu không xử lý. Đây là bài toán **schema evolution** — API versioning cùng các quy tắc tương thích (backward/forward) trình bày trong mục này là những giải pháp bổ trợ cho nhau [7, Ch.4].

3.2.1. Các chiến lược versioning

Ba chiến lược phiên bản hóa (versioning) nền tảng thường được áp dụng trong microservices:

- **URL path (ví dụ**: `/api/v1/questions`, `/api/v2/questions`): Cách tiếp cận này đơn giản và rất rõ ràng cho người sử dụng. Tuy nhiên, nó dẫn đến việc phải duy trì nhiều URL khác nhau, làm cho cấu hình routing trên hệ thống trở nên phức tạp hơn.
- **Query parameter (ví dụ**: `/questions?version=2`): Giúp hạn chế việc phải thay đổi cấu trúc URL cốt lõi. Mặc dù vậy, tham số này rất dễ bị bỏ sót trong quá trình tích hợp nếu người dùng không đọc kỹ tài liệu.
- **Header (ví dụ**: custom header `X-API-Version: 2`): Giữ cho URL hoàn toàn sạch sẽ (clean URL). Đổi lại, việc gọi thử nghiệm và debug trở nên khó khăn hơn vì yêu cầu cấu hình header thủ công, không trực quan như việc nhìn trực tiếp vào URL.

Theo Sam Newman, **URL path versioning** phổ biến nhất vì đơn giản và thân thiện với lập trình viên [4a, Ch.4]. Đây cũng là cách mà nhiều API public lớn (ví dụ Twitter) sử dụng. Ngoài ra còn một biến thể của header versioning đáng chú ý: *date-based versioning* — thay vì `v1`/`v2`, consumer ghim (pin) một ngày phát hành cụ thể qua header (Stripe dùng `Stripe-Version: 2024-12-18`); chúng ta sẽ gặp lại ví dụ Stripe ở phần phân tích KBM bên dưới.

Chiến lược thứ tư — **Media Type versioning** (hay **Content Negotiation versioning**) (`Accept: application/vnd.kbm.question.v2+json`) — phổ biến ở GitHub API: nhúng phiên bản vào header `Accept` (khác với custom header ở trên), cho phép version từng resource riêng lẻ thay vì toàn bộ API. Cần làm rõ rằng header `Accept` được dùng cho quá trình đàm phán nội dung (content negotiation — định dạng mà client kỳ vọng nhận về), trong khi `Content-Type` được dùng để mô tả định dạng của phần thân (body) trong request. Tuy nhiên, phương pháp này phức tạp hơn cho cả provider và consumer.

Bảng 3.2: *Tiêu chí lựa chọn — chọn chiến lược versioning nào?*

Tiêu chí	URL path	Query param	Header	Media Type
Dễ implement	⋆⋆⋆	⋆⋆⋆	⋆⋆	⋆
Cache-friendly	√ (URL khác nhau)	√ (URL khác nhau)	× (cần Vary header)	×

Tiêu chí	URL path	Query param	Header	Media Type
API Gateway routing	√ Dễ (path-based)	! Trung bình	! Trung bình	× Khó
Granularity	Toàn bộ API	Toàn bộ API	Toàn bộ API	Per resource
Dùng bởi	Twitter	Azure (`api-version`)	Stripe (`Stripe-Version`)	GitHub

Chính sách versioning — nguyên tắc vận hành:
Chọn chiến lược chưa đủ — cần **chính sách** rõ ràng cho cách version API:

1. **Khi nào cần nâng cấp phiên bản (version bump)?** — Chỉ khi có **thay đổi phá vỡ cấu trúc (breaking change)** (xóa trường dữ liệu, đổi kiểu dữ liệu, đổi hành vi). Thêm một trường dữ liệu (field) không bắt buộc → *không* cần phiên bản mới (tương thích ngược, backward compatible)
2. **Deprecation timeline** — Thông báo deprecation trước ít nhất **1 release cycle** qua các response header theo đúng định dạng chuẩn: `Deprecation: @1764547200` (Unix timestamp, RFC 9745 — ở đây là 2025-12-01, sớm hơn Sunset sáu tháng), `Sunset: Mon, 01 Jun 2026 00:00:00 GMT` (HTTP-date, RFC 8594), kèm `Link: <...>; rel="deprecation"` trỏ tới hướng dẫn chuyển đổi
3. **Chạy song song** — Phiên bản cũ và mới hoạt động đồng thời trong giai đoạn chuyển đổi (transition). Sử dụng Gateway routing để chuyển hướng dần lưu lượng truy cập (traffic) (Strangler Fig pattern — xem Ch.10)

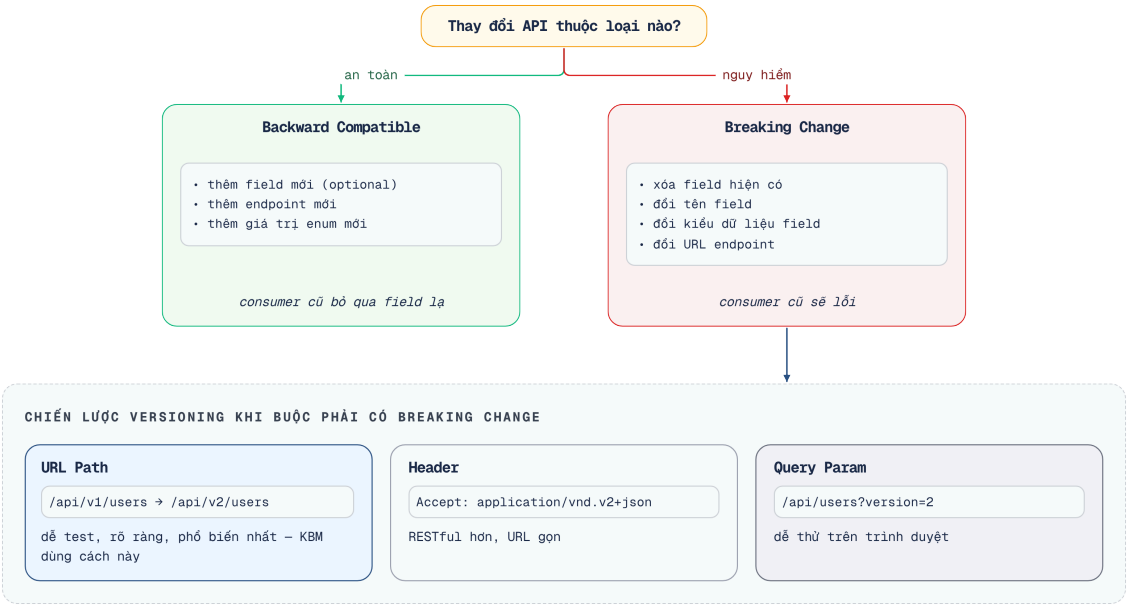
Chính sách versioning của KBM

Với KBM (LMS học thuật, team nhỏ), chiến lược đơn giản nhất: (1) **URL path versioning** (`/api/v1/questions`), (2) giữ **backward compatibility** bằng cách chỉ thêm trường (field), không xóa, (3) khi breaking change bắt buộc → tạo `/api/v2/` → chạy song song 1 học kỳ (semester) trước khi loại bỏ (sunset) v1. Phức tạp hơn không cần thiết cho hệ thống giáo dục nội bộ.

Schema Evolution — Thay đổi mà không breaking. Martin Kleppmann phân tích hai chiều compatibility [7, Ch.4]:

- **Tương thích ngược (Backward compatible)**: API phiên bản mới không làm gián đoạn các client cũ (ví dụ: thêm trường dữ liệu không bắt buộc — optional field — là thay đổi hợp lệ).
- **Tương thích xuôi (Forward compatible)**: Client phiên bản **cũ** vẫn xử lý được response do API phiên bản **mới** trả về — bỏ qua field lạ thay vì lỗi. Đây chính là điều kiện để Tolerant Reader (§3.9.2) hoạt động.

Quy tắc vàng khi thay đổi API:



Hình 3.2: *Quy tắc schema evolution — thay đổi nào là breaking change*

Tuy nhiên, các quy tắc tiến hóa dữ liệu này chỉ thực sự hiệu quả khi hệ thống đã có sẵn cơ chế định tuyến phiên bản rõ ràng ngay từ đầu.

KBM thiếu API versioning

KBM hiện chưa chuẩn hóa API versioning — nhiều endpoint nằm tại `/api/*` không có version prefix. Ở quy mô một consumer duy nhất, đây là **nợ kỹ thuật chấp nhận được (xem Bài học rút ra ở cuối chương) — nhưng phải trả trước khi mở rộng**: khi hệ thống thêm mobile app hoặc third-party integration, mỗi thay đổi API sẽ là breaking change cho consumer không kịp cập nhật. Stripe API — một trong những API được thiết kế tốt nhất — sử dụng date-based versioning (`2024-12-18`), cho phép consumer chỉ định cố định (pin) một version cụ thể. **Lộ trình chuyển đổi (Migration path)**: thêm tiền tố `/api/v1/` cho tất cả route hiện tại, cấu hình định tuyến trên Gateway, và lập tài liệu chính sách đánh phiên bản.

3.3. OpenAPI & API Documentation

Tại sao tài liệu (documentation) quan trọng? Chris Richardson nhấn mạnh rằng trong microservices, tài liệu API không phải là một "tùy chọn có thì tốt" (nice-to-have) mà là một **hợp đồng** (*contract*) giữa provider và consumer [2a, Ch.3]. Nếu tài liệu sai hoặc thiếu, lập trình viên phải đọc mã nguồn (source code) để hiểu API — điều này phá vỡ nguyên lý *Service Abstraction* [1].

OpenAPI Specification (Swagger). OpenAPI (trước đây gọi là Swagger) là chuẩn mô tả REST API dưới dạng YAML/JSON. Từ spec này, có thể tự động generate:

- Documentation UI (Swagger UI, ReDoc)
- Bộ thư viện phát triển máy khách (Client SDK: Java, TypeScript, Python)
- Mã khung máy chủ (Server stubs)
- Mã kiểm thử (Tests)

Trong Spring Boot (như LMS), sử dụng **SpringDoc OpenAPI** để tự động tạo spec từ code:

Listing 3.3: *Spring Boot controller với OpenAPI annotations*

```
// Annotations trên controller → auto-generate OpenAPI spec
@RestController
@RequestMapping("/api/questions")
@Tag(name = "Questions", description = "Quản lý câu hỏi SQL")
public class QuestionController {

    @GetMapping("/{id}")
    @Operation(summary = "Lấy chi tiết câu hỏi theo ID")
    @ApiResponse(responseCode = "200", description = "Thành công")
    @ApiResponse(responseCode = "404", description = "Không tìm thấy")
    public QuestionResponse getById(@PathVariable UUID id) {
        // ...
    }
}
```

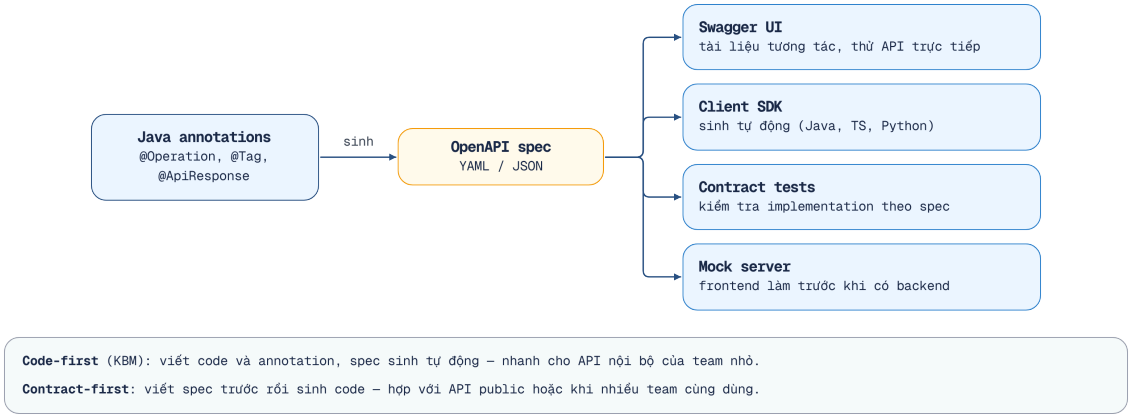
Kết quả: truy cập `/swagger-ui.html` → có giao diện người dùng (UI) tương tác để thử nghiệm (test) API trực tiếp.

Một quyết định quy trình quan trọng còn lại: viết spec trước hay viết code trước?

Contract-First vs Code-First

Có hai cách tiếp cận thiết kế API: **Contract-first** (viết OpenAPI spec trước, generate code sau) và **Code-first** (viết code trước, generate spec sau). Contract-first tốt hơn cho API public hoặc khi giao tiếp giữa nhiều nhóm phát triển (cross-team). Code-first nhanh hơn cho API internal trong các nhóm phát triển quy mô nhỏ. KBM sử dụng code-first (Spring Boot annotations → auto-generated docs) — hợp lý cho team nhỏ [1, Ch.7].

Hình 3.3 cho thấy toàn bộ chuỗi theo cách code-first: từ annotation trong code, qua OpenAPI spec, đến Swagger UI, client SDK, contract test và mock server.



Hình 3.3: *Pipeline từ code annotations đến Swagger UI và client SDK*

Cách làm này không chỉ giảm thiểu sức lao động thủ công mà còn giúp duy trì một single source of truth, đảm bảo tài liệu luôn đồng bộ với mã nguồn thực tế.

Documentation-as-Code

Coi API documentation như code: version control, review, automate. Nếu documentation nằm ngoài code (wiki, Confluence), nó sẽ nhanh chóng lỗi thời. SpringDoc/OpenAPI giải quyết vấn đề này bằng cách generate docs trực tiếp từ code.

3.4. DTO Pattern — tách Internal Model và External Contract

Vấn đề: Entity ≠ API Response. Sai lầm phổ biến trong microservices (và monolith) là trả về trực tiếp thực thể (entity) qua API:

Listing 3.4: Anti-pattern — trả entity trực tiếp qua API

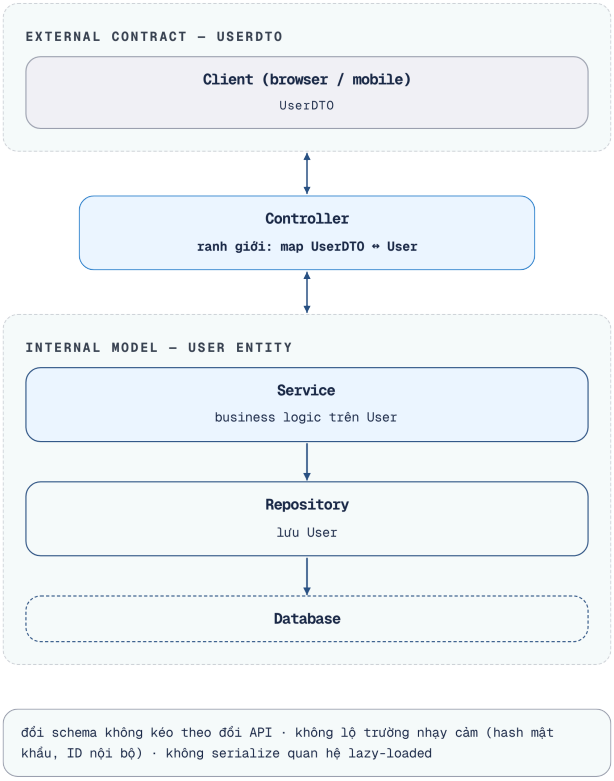
```
// SAI: Trả entity trực tiếp --- expose internal structure
@GetMapping("/questions/{id}")
public Question getById(@PathVariable UUID id) {
    return questionRepository.findById(id).orElseThrow();
}
```

Vấn đề:

- **Sự phụ thuộc (Coupling)**: Thay đổi lược đồ cơ sở dữ liệu (database schema) đồng nghĩa với việc thay đổi phản hồi API
- **Bảo mật (Security)**: Có nguy cơ để lộ (expose) các trường nhạy cảm (chuỗi băm mật khẩu, định danh nội bộ)
- **Hiệu năng (Performance)**: Thực thể có thể chứa các quan hệ tải lười (lazy-loaded) → dẫn đến lỗi N+1 truy vấn hoặc lỗi tuần tự hóa (serialization error)

3.4.1. DTO (Data Transfer Object)

DTO pattern tạo lớp đệm giữa internal model và external contract [2a], như minh họa ở Hình 3.4.



Hình 3.4: *DTO pattern — tách internal model và external contract*

Trong mã nguồn, lớp đệm này gồm DTO cho request, DTO cho response và một mapper chuyển đổi giữa entity và DTO (Listing 3.5):

Listing 3.5: *Request DTO, Response DTO và Mapper*

```
// ĐÚNG: Request DTO --- chỉ chứa field client cần gửi
public record CreateQuestionRequest(
    String title,
    String description,
    String solution,
    String difficulty
) {}

// ĐÚNG: Response DTO --- chỉ chứa field client cần nhận
public record QuestionResponse(
    UUID id,
    String title,
    String difficulty,
    int submitCount,
    double acceptRate,
    LocalDateTime createdAt
) {}

// Mapper chuyển đổi Entity ↔ DTO
@Mapper(componentModel = "spring")
public interface QuestionMapper {
    QuestionResponse toResponse(Question entity);
```

```
    Question toEntity(CreateQuestionRequest request);
}
```

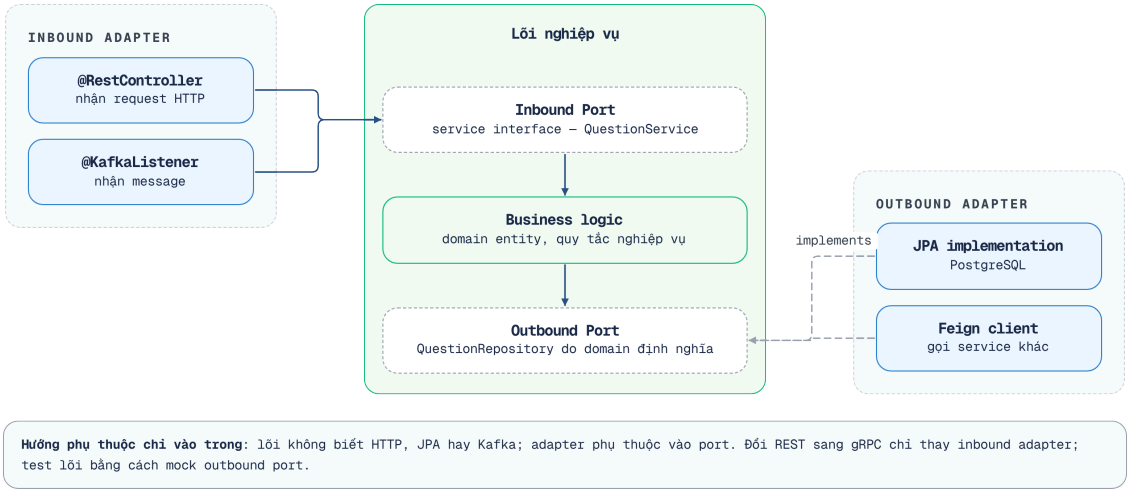
3.4.2. Lợi ích thực tế

- **Decoupling**: Thay đổi cấu trúc của entity (ví dụ: thêm cột trong cơ sở dữ liệu) sẽ không tự động làm thay đổi cấu trúc API trả về cho client.
- **Security**: Tránh được việc vô tình làm rò rỉ các trường dữ liệu nhạy cảm như mật khẩu hay các trường nội bộ (internal field) ra bên ngoài.
- **Performance**: Chỉ phản hồi những trường dữ liệu thực sự cần thiết, ngăn chặn tình trạng hệ thống tự động tải toàn bộ đồ thị đối tượng (lazy-load) không mong muốn.
- **Versioning**: Cho phép tạo ra các phiên bản response mới như `QuestionResponseV2` một cách độc lập mà không cần phải can thiệp hay chỉnh sửa entity gốc.
- **Validation**: Đưa các quy tắc kiểm tra tính hợp lệ (`@Valid`) lên mức DTO, giữ cho entity hoàn toàn tập trung vào biểu diễn dữ liệu cốt lõi.

Trong KBM, pattern này được áp dụng qua các package `dto/request/` và `dto/response/`, với MapStruct (`BaseMapper`) để tự động chuyển đổi. Các lớp cơ sở `BaseRequest` và `BaseResponse` trong shared library cung cấp format phản hồi chuẩn (`status`, `message`, `data` wrapper).

3.5. Hexagonal Architecture — tách biệt lõi nghiệp vụ khỏi hạ tầng

DTO pattern ở trên là một phần của kiến trúc rộng hơn mà Richardson mô tả chi tiết trong [2b, Ch.3 & Ch.13]: **Hexagonal Architecture** (còn gọi là Ports & Adapters, do Alistair Cockburn đề xuất). Đây là kiến trúc được đề xuất cho mỗi service trong microservices:



Hình 3.5: Hexagonal Architecture cho Core Service trong KBM

Nguyên tắc cốt lõi: Business logic **không phụ thuộc** vào các adapter — ngược lại, các adapter phụ thuộc vào business logic. Điều này có nghĩa:

- Đổi từ REST sang gRPC? → chỉ thay inbound adapter, business logic không đổi
- Đổi từ MySQL sang PostgreSQL? → chỉ thay outbound adapter
- Test business logic? → mock các outbound port, gọi trực tiếp vào inbound port

Các khái niệm của Spring Boot có thể được ánh xạ vào Hexagonal Architecture như sau:

- **Inbound Adapter**: Thành phần chịu trách nhiệm nhận request từ bên ngoài (ví dụ: `@RestController`, `@KafkaListener`).
- **Inbound Port**: API giao tiếp của business logic, thường là các service interface (ví dụ: `QuestionService`).
- **Business Logic**: Trái tim của hệ thống, bao gồm các domain entity và lớp hiện thực service (service implementation) xử lý nghiệp vụ cốt lõi.
- **Outbound Port**: Cách mà business logic định nghĩa yêu cầu giao tiếp với thế giới bên ngoài, là interface do chính lõi nghiệp vụ định nghĩa và chỉ nói bằng ngôn ngữ domain (ví dụ `QuestionRepository` tự viết). Lưu ý `JpaRepository` **không phải** outbound port — nó thuộc về adapter; để port sạch, domain không import bất kỳ type nào của Spring Data.
- **Outbound Adapter**: Các thành phần thực hiện kết nối thực tế ra hệ thống bên ngoài, như JPA implementation, Feign client, hay Kafka producer.

Mô hình Hexagonal Architecture mang lại sự phân tách rạch ròi về mặt mã nguồn, hỗ trợ xây dựng hệ thống mềm dẻo.

Iceberg Principle

Hexagonal Architecture là hiện thực hóa của **Iceberg Principle** (Richardson [2b, Ch.4]): Inbound Ports (API) là phần nổi — nhỏ, ổn định. Phần chìm bao gồm Business Logic (cốt lõi nghiệp vụ) và Outbound Adapters (hạ tầng kỹ thuật). Hexagonal tách biệt chúng để hạ tầng linh hoạt thay thế mà không ảnh hưởng lõi nghiệp vụ. DTO pattern là cơ chế đảm bảo entity (phần chìm) không bị để lộ (expose) qua API (phần nổi).

Ví dụ minh họa sự khác biệt giữa Port (Interface) và Adapter (Implementation):

```
// Outbound Port (Business Logic)
public interface QuestionRepository {
    Question save(Question q); // Domain model
}

// Outbound Adapter (Infrastructure)
@Repository
public class JpaQuestionAdapter implements QuestionRepository {
    @Override
    public Question save(Question q) {
        return springDataRepo.save(EntityMapper.toJpa(q)); // Database model
    }
}
```

3.6. Contract Testing — kiểm thử hợp đồng API

3.6.1. Vấn đề: API thay đổi làm lỗi máy khách (crash consumer)

Tài liệu OpenAPI mô tả diện mạo của API — nhưng không đảm bảo consumer và provider thực sự đồng ý về ý nghĩa của từng trường dữ liệu (field), mã trạng thái (status code), và các trường hợp biên (edge case). Trong microservices, mỗi service là một independent deployment unit: khi `kbm-core` thay đổi cấu trúc phản hồi (response schema), `kbm-academic` (consumer) có thể gặp sự cố trong quá trình thực thi (runtime crash) — và nhóm phát triển sẽ chỉ phát hiện sau khi deploy lên môi trường staging hoặc production.

Hợp đồng API và lịch học

Sự thay đổi API không báo trước giống như việc Phòng Đào tạo (Provider) tự ý đổi giờ học môn Cơ sở dữ liệu từ sáng sang chiều mà không báo cho sinh viên (Consumer). Dù môn học vẫn diễn ra (các bài kiểm thử đơn vị vẫn đạt), nhưng sinh viên không thể tham gia vì kẹt lịch khác (máy khách bị lỗi/crash). Contract Testing giúp Phòng Đào tạo phải "hỏi" sinh viên xem có ai bị kẹt lịch không trước khi chính thức ban hành thay đổi.

Các bài kiểm thử đơn vị (unit test) và kiểm thử tích hợp (integration test) truyền thống không giải quyết được vấn đề này vì chúng kiểm thử từng dịch vụ độc lập. Cần một cơ chế để **hệ thống cung cấp biết trước khi deploy liệu thay đổi của nó có làm hỏng (break) bất kỳ máy khách nào không**.

3.6.2. Consumer-Driven Contract Testing

Consumer-Driven Contract Testing (CDCT) giải quyết bằng cách để consumer viết "contract" — tập hợp kỳ vọng cụ thể về API của provider. Provider sau đó verify contract này trong test suite của hệ thống cung cấp trước khi deploy:

Luồng hoạt động chuẩn của Consumer-Driven Contract Testing (tiêu biểu là **Pact**) diễn ra qua năm bước cốt lõi sau:

1. **Consumer team** viết kiểm thử mô tả chi tiết request gửi đi và response kỳ vọng nhận lại từ provider.
2. **Framework** (Pact) chạy kiểm thử phía consumer với mock server và tự động sinh ra *contract file* (Pact JSON).
3. Thông qua **CI pipeline**, contract file được phát hành (publish) lên kho lưu trữ hợp đồng trung tâm (*Pact Broker*).
4. **Provider team** trong CI tự động kéo các contract từ Broker về và phát lại (replay) request vào provider thực để xác minh phản hồi.
5. Nếu thay đổi của provider làm vỡ bất kỳ hợp đồng nào đang hiệu lực, **CI pipeline** lập tức chặn deploy trước khi lỗi lọt lên production.

Với **Spring Cloud Contract**, chiều thực thi hơi khác dù cùng mục tiêu bảo vệ consumer: hợp đồng được viết bằng Groovy DSL hoặc YAML (do consumer đề xuất qua pull request vào kho của provider hoặc kho hợp đồng chung); plugin trên phía provider tự sinh bộ kiểm thử JUnit (`@SpringBootTest`) để xác minh API thật, đồng thời đóng gói WireMock stub JAR phát hành lên kho artifact (Maven/Nexus) cho consumer tải về chạy kiểm thử tích hợp.

Hai framework phổ biến trong Java/Spring ecosystem:

Bảng 3.3: *So sánh Pact và Spring Cloud Contract*

Khía cạnh	Pact	Spring Cloud Contract
Ngôn ngữ	Polyglot (Java, JS, Go, Python...)	Java/Spring (JVM-only)
Contract format	Pact JSON	Groovy DSL / YAML
Broker	Pact Broker (self-hosted/cloud)	Artifact repo (Maven/Nexus)
Tích hợp Spring Boot	Pact JVM + MockMvc	Native (`@SpringBootTest`)
Phù hợp khi	Multi-language, nhiều consumer	Stack thuần Java/Spring

3.6.3. Áp dụng cho KBM

Trong KBM, `kbm-academic` là consumer chính của `kbm-core` (truy vấn câu hỏi, vd. `GET /api/questions/batch`). Nếu nhóm phát triển thêm hoặc đổi tên trường dữ liệu (field) trong phản hồi của `kbm-core`, sẽ không có cảnh báo nào xuất hiện cho đến khi `kbm-academic` nhận được phản hồi không đúng cấu trúc tại thời điểm thực thi (runtime).

Kịch bản minh hoạ: `kbm-core` đổi `"title"` → `"name"` trong phản hồi câu hỏi. `kbm-academic` đọc trường `title` → nhận `null` → trang bài tập hiện câu hỏi không có tên (hoặc NullPointerException nếu code không kiểm tra null). Không có lỗi biên dịch (compile error), các bài kiểm thử đơn vị (unit test) vẫn vượt qua thành công.

Với Contract Testing (Spring Cloud Contract):

```
// contract định nghĩa bởi kbm-academic (consumer)
Contract.make {
  request {
    method GET()
    url "/api/questions/123"
  }
  response {
    status 200
    body([id: "123", title: "Truy vấn JOIN cơ bản", difficulty: "EASY"])  //
consumer expect field title
    headers { contentType(applicationJson()) }
  }
}
```

Khi `kbm-core` chạy xác minh hợp đồng này bằng trường dữ liệu `name` thay vì `title` → kiểm thử thất bại (test fail) → hệ thống CI chặn đứng quá trình deploy → nhóm phát triển phát hiện lỗi trước khi đưa lên môi trường thực tế (production).

Khi nào dùng Contract Testing?

Contract Testing mang lại giá trị khi: (1) có từ 2 nhóm phát triển trở lên sở hữu các service giao tiếp với nhau, (2) chu kỳ triển khai khác biệt (consumer và provider deploy độc lập), (3) đã có unit test và integration test cơ bản. Với nhóm phát triển quy mô nhỏ và deploy cùng lúc, chi phí thiết lập (setup) Pact Broker có thể không mang lại hiệu quả tương xứng — ưu tiên OpenAPI contract + integration test trước.

3.7. GraphQL — khi REST không đủ linh hoạt

REST là chuẩn phổ biến nhất cho các API trong microservices, nhưng không phải lựa chọn duy nhất. **GraphQL** (Facebook, 2012; open-sourced 2015) giải quyết một số hạn chế của REST bằng cách cho phép client **chọn chính xác data cần lấy**.

Vấn đề mà GraphQL giải quyết. Với REST, mỗi endpoint trả về *fixed structure*. Client lấy nhiều hơn hoặc ít hơn data cần thiết. Cụ thể, GraphQL nhắm vào ba vấn đề kinh điển:

- **Over-fetching**: API trả về quá nhiều dữ liệu không cần thiết. Ví dụ, `GET /questions/1` trả về 20 trường, nhưng mobile chỉ cần `title` và `difficulty`. Với GraphQL, client chỉ yêu cầu đúng những trường cần thiết: `{ question(id:1) { title difficulty } }`.

- **Under-fetching**: API không trả đủ dữ liệu, buộc phải gọi nhiều lần. Ví dụ, dashboard cần thông tin question, submissions và user, dẫn đến 3 lệnh gọi API. GraphQL cho phép lấy tất cả trong 1 query: `{ question { title submissions { score } user { name } } }`.
- **Multiple round trips**: Việc gọi nhiều endpoint như trên gây ra nhiều request qua mạng (đặc biệt tốn kém về độ trễ trên mobile). GraphQL gộp thành 1 request và 1 response duy nhất, tối ưu đáng kể độ trễ.

3.7.1. REST vs GraphQL — so sánh

Bảng 3.4 so sánh hai cách tiếp cận theo bảy tiêu chí.

__Bảng 3.4:__ REST vs GraphQL — so sánh chi tiết

Tiêu chí	REST	GraphQL
Endpoint	Nhiều endpoint (`/questions`, `/submissions`, `/users`)	Một endpoint (`/graphql`)
Hình dáng dữ liệu (Data shape)	Máy chủ quyết định cấu trúc (structure) trả về	Máy khách quyết định các trường (field) cần lấy
Versioning	URL versioning (`/v1/`, `/v2/`) hoặc header	Schema evolution — thêm trường mới, đánh dấu ngừng dùng (deprecate) trường cũ
Caching	HTTP caching tự nhiên (GET, ETag, Cache-Control)	Phức tạp hơn (dùng POST request, cần Apollo Cache hoặc tương tự)
Tooling	Swagger/OpenAPI — trưởng thành, phổ biến rộng rãi	GraphQL Playground, Apollo DevTools — mạnh mẽ nhưng hệ sinh thái nhỏ hơn
Learning curve	Thấp (HTTP verbs, status codes)	Trung bình (schema language, resolvers, N+1 problem)
Xử lý lỗi	Mã trạng thái HTTP (400, 404, 500)	Operation đã thực thi trả `200 OK` kèm mảng `errors` (kể cả khi có field lỗi); lỗi transport/request hỏng/xác thực vẫn dùng 4xx/5xx — ngữ nghĩa status code mờ hơn REST

3.7.2. N+1 Problem — thách thức chính của GraphQL

Với GraphQL, một truy vấn lồng nhau có thể khiến server thực thi N+1 câu SQL:

```
# Query: lấy tất cả questions và submissions cho mỗi question
{ questions { title submissions { score } } }

# Server thực thi:
# 1. SELECT * FROM questions (N questions)
# 2. Với MỖI question: SELECT * FROM submissions WHERE question_id = ?
# → N+1 queries! (1 cho questions + N cho submissions)
```

Giải pháp: DataLoader pattern — gom N truy vấn thành 1: `SELECT * FROM submissions WHERE question_id IN (1, 2, ..., N)`. Facebook open-sourced DataLoader library cho mục đích này. Điểm khác biệt với REST: trong REST, server quyết định trước query structure nên lập trình viên có thể tối ưu sẵn từng endpoint (dù N+1 vẫn xảy ra ở tầng ORM nếu bất cẩn — xem §3.4); trong GraphQL, truy vấn do client quyết định nên server bắt buộc phải xử lý N+1 một cách tường minh.

Schema-First Approach. GraphQL server nào cũng cần schema ở runtime, nhưng quy trình phát triển có thể theo hướng *schema-first* (viết schema trước rồi implement resolver) hoặc *code-first* (sinh schema từ code/annotation). Sách chọn schema-first cho ví dụ KBM — tương tự contract-first cho REST (OpenAPI spec trước code), có lợi cho governance và review hợp đồng giữa các team:

Listing 3.6: *GraphQL schema definition cho KBM*

```
type Question {
  id: ID!
  title: String!
  difficulty: Difficulty!
  submissions: [Submission!]!
  createdBy: User!
}

type Query {
  question(id: ID!): Question
  questions(difficulty: Difficulty, limit: Int = 20): [Question!]!
}

enum Difficulty { EASY MEDIUM HARD }
```

Lược đồ (schema) này vừa là tài liệu (documentation), vừa là hợp đồng (contract), vừa là hệ thống kiểu dữ liệu (type system) — tính năng GraphQL introspection cho phép các công cụ (tool) tự động sinh ra mã nguồn máy khách (client code) và tài liệu.

3.7.3. Khi nào chọn REST, khi nào chọn GraphQL?

Bảng 3.5 gợi ý lựa chọn theo từng kịch bản sử dụng.

Bảng 3.5: *Khi nào chọn REST, khi nào chọn GraphQL*

Kịch bản	Khuyến nghị	Lý do
Service-to-service (backend)	REST hoặc gRPC	Đơn giản, dễ lưu đệm (cacheable), các nhóm kiểm soát cả hai phía
Backend-for-Frontend (BFF) (mobile)	GraphQL	Ứng dụng di động cần các truy vấn linh hoạt (flexible query), giảm số vòng giao tiếp qua mạng (round trip)
Public API	REST	HTTP standards, caching, documentation ecosystem
Dashboard aggregation	GraphQL	1 truy vấn thay vì 5–10 lệnh gọi REST
CRUD đơn giản	REST	Chi phí bổ sung (overhead) của GraphQL không tương xứng với lợi ích mang lại
Real-time (subscriptions)	GraphQL + WebSocket	Native support cho subscriptions

KBM hiện tại: REST vẫn là lựa chọn phù hợp cho LMS core vì đơn giản, dễ debug và được OpenAPI hỗ trợ tốt. Tuy nhiên, KBM v2.0 đã có hệ sinh thái API phức tạp hơn: Network Lab dùng nhiều protocol để phục vụ bài học tích hợp hệ thống; DevOps Lab dùng Go/Chi cho router và API vận hành; các luồng chấm bài có thể đi qua Kafka thay vì HTTP đồng bộ. GraphQL sẽ phát huy giá trị nếu KBM bổ sung: (1) **mobile app** (ứng dụng di động) — đòi hỏi các truy vấn linh hoạt cho nhiều kích thước màn hình khác nhau, (2) **teacher dashboard** (trang quản trị của giảng viên) — cần tổng hợp dữ liệu (aggregate data) từ nhiều dịch vụ chỉ thông qua một yêu cầu, (3) **third-party**

API (API dành cho bên thứ ba) — người dùng bên ngoài cần kiểm soát chính xác lượng dữ liệu họ nhận được.

Điều quan trọng cần nhớ: **GraphQL không phải là sự thay thế cho REST** — chúng giải quyết vấn đề khác nhau. REST tối ưu cho giao tiếp giữa các service (đơn giản, cache được, theo chuẩn); GraphQL tối ưu cho các API hướng máy khách (client-facing API — linh hoạt, một endpoint, có kiểu dữ liệu). Nhiều hệ thống dùng cả hai: REST giữa các service, GraphQL cho tầng BFF (Richardson [2b, Ch.8]).

⚠ Sai lầm thường gặp

1. **Trả entity trực tiếp qua API** — Không dùng DTO, expose toàn bộ internal structure (password hash, internal IDs, lazy-loaded relations). Hậu quả: thay đổi schema = breaking change cho mọi consumer, rủi ro bảo mật. *Phòng tránh*: luôn dùng DTO pattern (§3.4) — tách internal model khỏi external contract.
2. **Không thống nhất naming convention từ đầu** — Trộn lẫn giữa danh từ số ít và số nhiều, hoặc mix kebab-case với camelCase trong cùng hệ thống. Hậu quả: khi có từ 3 consumer trở lên (mobile, third-party, internal), việc thay đổi quy tắc đặt tên (naming) sẽ dẫn đến hàng loạt breaking change. *Cách phòng tránh*: lựa chọn quy ước (ví dụ: plural + kebab-case) và bắt buộc tuân thủ (enforce) ngay từ các Pull Request đầu tiên.
3. **Bỏ qua error format chuẩn** — Mỗi dịch vụ trả về lỗi theo định dạng riêng, mỗi lập trình viên tự viết logic xử lý lỗi. Hậu quả: consumer phải xử lý (handle) nhiều định dạng khác nhau, khiến quá trình gỡ lỗi (debugging) khó khăn. *Phòng tránh*: dùng `GlobalExceptionHandler` với `@ControllerAdvice` và error response format thống nhất cho toàn hệ thống (§3.1).

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

Xem minh họa động tại companion web:

- **Thiết kế REST API**: rest-api-explorer.html
- **GraphQL và REST**: graphql-vs-rest.html

3.8. Ranh giới của chương này

Chương 3 dừng ở *hợp đồng API*: tài nguyên được đặt tên ra sao, lỗi được phát biểu thế nào, và hợp đồng tiến hóa mà không phá client bằng cách nào. Hai câu hỏi tự nhiên nảy ra từ đây lại thuộc về chương khác, và sách trả lời chúng ở đúng chỗ có đủ ngữ cảnh:

- **"REST có phải lựa chọn duy nhất cho giao tiếp nội bộ không?"** — gRPC, Protobuf và tiêu chí chọn giữa hai bên được trình bày ở **Chương 4**, kèm bảng so sánh chi tiết và phân tích vì sao lõi KBM vẫn giữ REST/JSON.
- **"Frontend làm sao gọi tới hàng chục service?"** — API Gateway và biến thể BFF được trình bày ở **Chương 8**, nơi chúng đi cùng routing, xác thực tại biên và rate limiting.

3.9. Case study: KBM

Thiết kế API của KBM lớn lên theo nhu cầu từng đợt, nên có chỗ làm tốt và có chỗ chưa nhất quán — đúng như phần lớn hệ thống thật.

3.9.1. Hiện trạng

Phân tích source code KBM cho thấy một số quyết định API design thực tế — một số tốt, một số là *phản mẫu* (anti-pattern) do phát triển hữu cơ (organic growth):

Điểm mạnh:

- DTO pattern được áp dụng nhất quán (request/response separation)
- Swagger/OpenAPI tích hợp sẵn
- JSON response format chuẩn (`BaseResponse<T>` wrapper)
- KBM giữ REST/OpenAPI làm contract chính cho LMS core, đồng thời mở rộng Network Lab bằng SOAP/REST/gRPC/JNP và DevOps Lab bằng Go/Chi router. Vì vậy, case study hiện tại là **multi-protocol có chủ đích**, không còn là một hệ thống REST thuần.

Điểm cần cải thiện (từ khảo sát mã nguồn KBM):

Inconsistent naming. Hiện tại, hệ thống đang trộn lẫn giữa `/question` (danh từ số ít) với `/users` (số nhiều) và sử dụng `/submit-history` (kebab-case). Cần chuẩn hóa toàn bộ để nhất quán dùng danh từ số nhiều kết hợp với kebab-case.

Không có API versioning. Tất cả các endpoint đang được đặt chung tại `/api/*`. Giải pháp khắc phục là bổ sung thêm tiền tố phiên bản như `/api/v1/*` để dễ dàng nâng cấp trong tương lai.

Error response không nhất quán. Mã lỗi trả về đang không đồng nhất, điển hình là sự khác biệt giữa `ServerResponse.description` và `JwtUtil.message`. Nên chuẩn hóa một định dạng phản hồi lỗi duy nhất thông qua `GlobalExceptionHandler`.

Hard delete only. Dữ liệu khi xóa đang bị xóa vĩnh viễn khỏi hệ thống thay vì soft-delete. Việc bổ sung cơ chế soft-delete sẽ giúp lưu vết (audit trail) tốt hơn cho mục đích tra cứu sau này.

SQL Judge worker contract chưa chuẩn hóa. Dịch vụ điều phối `kbm-judge-sql` chuyển yêu cầu tới các sandbox worker như `kbm-sandbox-mysql`, `kbm-sandbox-sqlserver` nhưng cấu trúc `JudgeRequest` và `JudgeResult` chưa có hợp đồng (contract) rõ ràng. Cần xây dựng DTO và OpenAPI chuẩn dùng chung cho tất cả các `kbm-sandbox-*`, đi kèm với cơ chế đánh phiên bản và idempotency key.

3.9.2. Bài học rút ra

Những sự không nhất quán này không phải "lỗi" — chúng là kết quả tự nhiên khi hệ thống phát triển nhanh với team nhỏ, qua nhiều vòng lặp phát triển. Bài học quan trọng:

1. **Naming convention nên được thống nhất sớm** — chi phí sửa sau này tăng theo số lượng consumer. Khi API chỉ có một máy khách tiêu thụ (frontend), việc đổi tên rất dễ dàng. Tuy nhiên, khi phục vụ từ ba máy khách trở lên (ứng dụng di động, bên thứ ba, dịch vụ nội bộ), việc đổi tên đồng nghĩa với một thay đổi phá vỡ cấu trúc (breaking change).
2. **Error format chuẩn hóa từ đầu** — `GlobalExceptionHandler` với `@ControllerAdvice` giải quyết phần lớn các trường hợp lỗi phổ biến. Trong KBM, handler tồn tại nhưng Gateway xử lý lỗi JWT riêng (vì được phát triển độc lập) → hai format khác nhau.
3. **API versioning chỉ cần khi có nhiều consumer** — với team sở hữu cả provider và consumer, implicit versioning (deploy cùng nhau) là đủ. Không nên phức tạp hóa hay thiết kế quá mức.

Ở phía tiêu thụ, cách client xử lý dữ liệu trả về cũng quyết định độ ổn định của hệ thống.

Tolerant Reader

Martin Fowler đề xuất pattern "Tolerant Reader": consumer nên bỏ qua các trường (field) không biết và chỉ đọc những trường cần thiết. Nhờ consumer chịu được phiên bản mới của provider (forward-compatible), provider có thể thêm trường mới mà không gây lỗi (breaking) cho các consumer — nhìn từ phía provider, đó chính là **backward compatibility tự nhiên** [W6].

3.9.3. Phục vụ nhiều loại client và tích hợp judge ngoài — bài học mở rộng

Phần case study ở trên tập trung vào API giữa các service nội bộ. Nhưng KBM còn đặt ra hai câu hỏi API design mà nhiều hệ thống thực tế phải trả lời: khi backend phục vụ *nhiều loại client khác stack*, và khi nên *tự thiết kế API* hay *chấp nhận hợp đồng của một dịch vụ ngoài*.

Một backend, nhiều loại client

KBM có hai client tiêu thụ **cùng một backend và cùng một hệ định danh** nhưng phục vụ hai ngữ cảnh khác nhau:

- `kbm-web` **(React/TypeScript)**: client luyện tập hàng ngày, chạy trên trình duyệt bất kỳ, nộp bài ở chế độ luyện tập.
- `kbm-exam-client` **(desktop thick client)**: client thi trên phòng máy, siết chặt môi trường thi (lockdown), chỉ kết nối từ IP phòng thi.

Cả hai dùng chung REST API và cùng JWT do `kbm-auth` cấp (SSO) — *không* có "API riêng cho desktop". Đây là một nguyên tắc thiết kế quan trọng: API là hợp đồng **trung lập với client**. Khi contract ổn định và tách khỏi chi tiết client (nhờ DTO, chính sách versioning, error format chuẩn ở §3.1–3.4), việc thêm một loại client mới — desktop, mobile, hay third-party — không buộc phải thiết kế lại backend. Khác biệt giữa web và desktop (lockdown, ràng IP, tín hiệu proctoring) nằm ở *client* và ở phần kiểm soát truy cập phía server, không phải ở hình dạng API.

API trung lập với client

Một API tốt không "biết" ai đang gọi nó. `kbm-exam-client` (desktop) và `kbm-web` (React) gửi cùng request, cùng JWT, nhận cùng response — backend chỉ thấy một định danh hợp lệ và một hành động. Khi các máy khách không đồng nhất (heterogeneous client) chia sẻ chung một hợp đồng và hệ định danh (identity), chi phí thêm máy khách mới gần như chỉ là chi phí viết máy khách đó. Ngược lại, nếu mỗi máy khách có một API riêng, số hợp đồng (contract) phải bảo trì tăng theo số loại máy khách.

API bộ chấm code: tự thiết kế vs tích hợp hợp đồng OSS

Module chấm bài lập trình đa ngôn ngữ (`kbm-judge-code`) minh họa một lựa chọn API *ngược* với phần còn lại của chương. Thế hệ đầu (Gen 1) là bộ chấm tự viết cho C/C++ và Java — nhóm tự định nghĩa toàn bộ API surface: `JudgeRequest`/`JudgeResult` riêng, format verdict riêng, hợp đồng do nhóm phát triển tự kiểm soát hoàn toàn. Thế hệ hiện tại (Gen 2) tích hợp **DOMjudge** — một online judge mã nguồn mở chuẩn thi ICPC.

Điểm mấu chốt về API design: khi áp dụng DOMjudge, KBM **không còn tự thiết kế API chấm code** mà phải tiêu thụ hợp đồng API *đã có sẵn* của DOMjudge (các endpoint REST submit/judging, định dạng verdict, kênh trả kết quả). Bài toán đổi từ "thiết kế một API tốt" sang "tích hợp API của người khác cho tốt": tiến hành ánh xạ (map) hợp đồng từ hệ thống bên ngoài về các DTO nội bộ, đặt một adapter ở ranh giới (boundary) để che giấu chi tiết DOMjudge khỏi phần còn lại của hệ thống

(Anti-Corruption Layer — xem §2.3 và Ch.10), và đảm bảo máy khách tiêu thụ (consumer) nội bộ chỉ nhận được một `JudgeResult` chuẩn hóa. Đây là một quyết định **'xây hay mua' (build-vs-buy)** điển hình; khía cạnh giao tiếp đồng bộ và bảo mật/sandbox được phân tích tiếp ở Ch.4 và Ch.9.

3.10. Tổng kết

Thiết kế API chuẩn mực đảm bảo sự ổn định cho toàn bộ kiến trúc microservices.

API versioning và schema evolution là bài toán không thể tránh khỏi khi hệ thống phát triển. Ba chiến lược versioning (URL path, query param, header) mỗi cách có trade-off riêng — URL path phổ biến nhất vì thân thiện với lập trình viên. Quy tắc tương thích ngược (backward compatibility) giúp thay đổi API mà không làm lỗi máy khách (consumer crash).

OpenAPI/Swagger biến tài liệu (documentation) thành mã nguồn (code) — tự động, luôn cập nhật, và có thể thử nghiệm (test) trực tiếp. Mẫu thiết kế DTO (DTO pattern) tách biệt mô hình nội bộ (internal model) và hợp đồng bên ngoài (external contract) — điều kiện cần để quá trình tiến hóa API (API evolution) không gắn chặt vào lược đồ cơ sở dữ liệu (database schema). Mở rộng nguyên tắc tách biệt đó lên toàn service, Hexagonal Architecture (Ports & Adapters) cô lập lõi nghiệp vụ khỏi chi tiết giao thức và lưu trữ bên ngoài.

Contract Testing (Pact, Spring Cloud Contract) đưa khái niệm "API là hợp đồng" lên một tầng cao hơn: consumer định nghĩa kỳ vọng, provider xác minh trước khi deploy — giúp phát hiện sớm các thay đổi phá vỡ cấu trúc (breaking change) tại môi trường CI thay vì môi trường thực tế (production). Bên cạnh REST cho giao tiếp liên dịch vụ và API chuẩn hóa, GraphQL cung cấp lựa chọn bổ trợ hiệu quả cho tầng hướng máy khách (BFF, dashboard tổng hợp) khi cần truy vấn linh hoạt, miễn là kiểm soát tốt bài toán N+1 bằng DataLoader.

Phân tích KBM cho thấy: Thiết kế API (API design) "đủ tốt" là hoàn toàn khả thi cho nhóm nhỏ (team nhỏ), nhưng cần ý thức chuẩn hóa từ sớm (quy tắc đặt tên, định dạng lỗi, chính sách phiên bản) để tránh tích lũy nợ kỹ thuật (technical debt).

3.11. Đọc thêm

Sách tham khảo chính:

1. [1] Thomas Erl, *SOA: Analysis and Design for Services and Microservices*, 2nd Ed. — Ch.7: Service API Design, Service Contract
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.3: Interprocess Communication, REST API Design
3. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.3: Architecture Fundamentals, Hexagonal Architecture; Ch.13: Service Design with Ports & Adapters
4. [4a] Sam Newman, *Building Microservices* — Ch.4: Integration, REST best practices
5. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.4: Encoding and Evolution, Schema Evolution

Nguồn trực tuyến:

- [W6] Martin Fowler, "TolerantReader" — martinfowler.com/bliki/TolerantReader.html
- Leonard Richardson, "Richardson Maturity Model" — martinfowler.com/articles/richardsonMaturityModel.html
- OpenAPI Specification 3.1 — spec.openapis.org
- Pact documentation — docs.pact.io (Consumer-Driven Contract Testing)

- Spring Cloud Contract — docs.spring.io/spring-cloud-contract

Bài tập: Xem phần Bài tập & case study ở cuối sách để luyện tập:

- **3-0**: Ôn tập: các nguyên tắc thiết kế API ([L1])
- **3-1**: Case study: Stripe API — tại sao được coi là "gold standard"? ([L2])
- **3-2**: Debugging: Sự cố breaking change của API ([L3])

Tài liệu tham khảo

[1] Erl, T. (2017). *SOA Analysis & Design for Services and Microservices* (2nd ed.). Prentice Hall.

[2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[2b] Richardson, C. (2025). *Microservices Patterns: With Examples in Java* (2nd ed.; bản MEAP, truy cập 07/2026). Manning Publications.

[4a] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.

[7] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.